Profili differenti per un Assistente AI

Prof. Ing. Loris Penserini, PhD

https://orcid.org/0009-0008-6157-0396

Overview

Parleremo di:

- Introduzione ai modelli LLM in pratica con TensorFlow
- Assistente AI con «Ollama»
- IoT con smart plugs: Shelly & Tapo
- Comandi vocali con «pyttsx3» e Ollama
- UI per Python con QT Designer



Creare un assistente AI con capacità di giocare ruoli diversi

Costruire GUI in Python

Non esistono tante soluzioni che soddisfino tutte queste features:

- appoggiarsi su **librerie grafiche native**: questo è un vantaggio in termini di efficienza dell'applicazione grafica, che sfrutterà appieno le capacità del sistema
- **IDE per lo sviluppo**: l'esistenza di uno strumento IDE risulta fondamentale per abbattere i tempi di sviluppo e test.
- il **tipo di licenza** offerta dal framework opensource
- **Widget**: è importante conoscere quali strumenti visuali mette a disposizione il framework (es. bottoni, menu, canvas, ecc...)
- **Cross-platform:** totalmente portabili, conferiscono (quasi) sempre lo stesso aspetto a tutte le applicazioni, indipendentemente dal sistema operativo che le esegue

PyQT e QT Designer

PyQT soddisfa tutte le caratteristiche elencate prima…

```
pip install PyQt6
pip install pyqt6-tools
```

Nelle versioni più recenti di PySide6, Qt Designer viene installato automaticamente quando si installa PySide6 con pip. Dopo l'installazione, è possibile eseguire Qt Designer dalla riga di comando utilizzando il launcher integrato:

```
pyside6-designer
```

QT Designer

https://www.pythonguis.com/installation/install-qt-designer-standalone/

https://pypi.org/project/PyQt5Designer/

Come si lavora con Qt Designer

- Apri Qt Designer e scegli un tipo di finestra (Main Window, Dialog, Widget).
- Trascina (drag&drop) i widget nella finestra.
- Organizza i widget usando layout (verticale, orizzontale, griglia).
- Modifica le proprietà (testo, font, colori, dimensioni).
- Salva il file come <nome_file>**.ui**

Trasformare il file *.ui in *.py:

pyside6-uic nome.ui -o nome.py

Esempio con QT & Python

(problema) Creare una GUI per sommare due numeri decimali al click di un bottone.

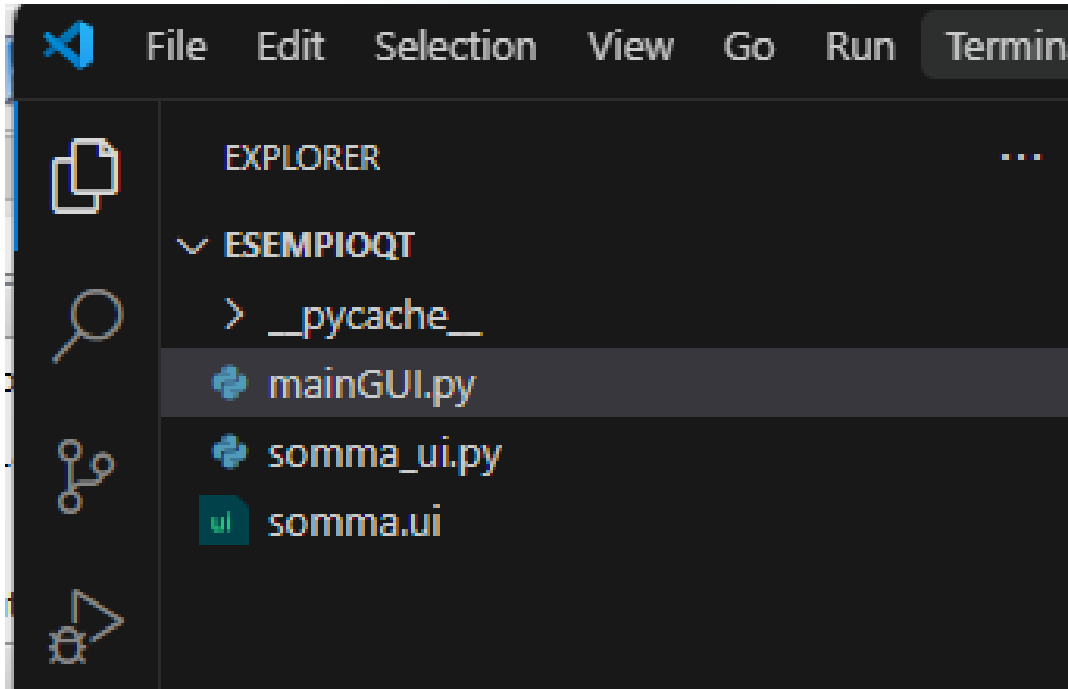
(soluzione) Architettura del progetto:

- «**somma.ui**»: GUI realizzata con QT Designer
- «**somma_ui.py**»: trasformazione della GUI in codice Python (**pyside6-uic somma.ui -o somma_ui.py**)
- «**mainGUI.py**»: classe che istanzia 'somma_ui.py' e crea la logica di funzionamento.

Esempio con QT & Python

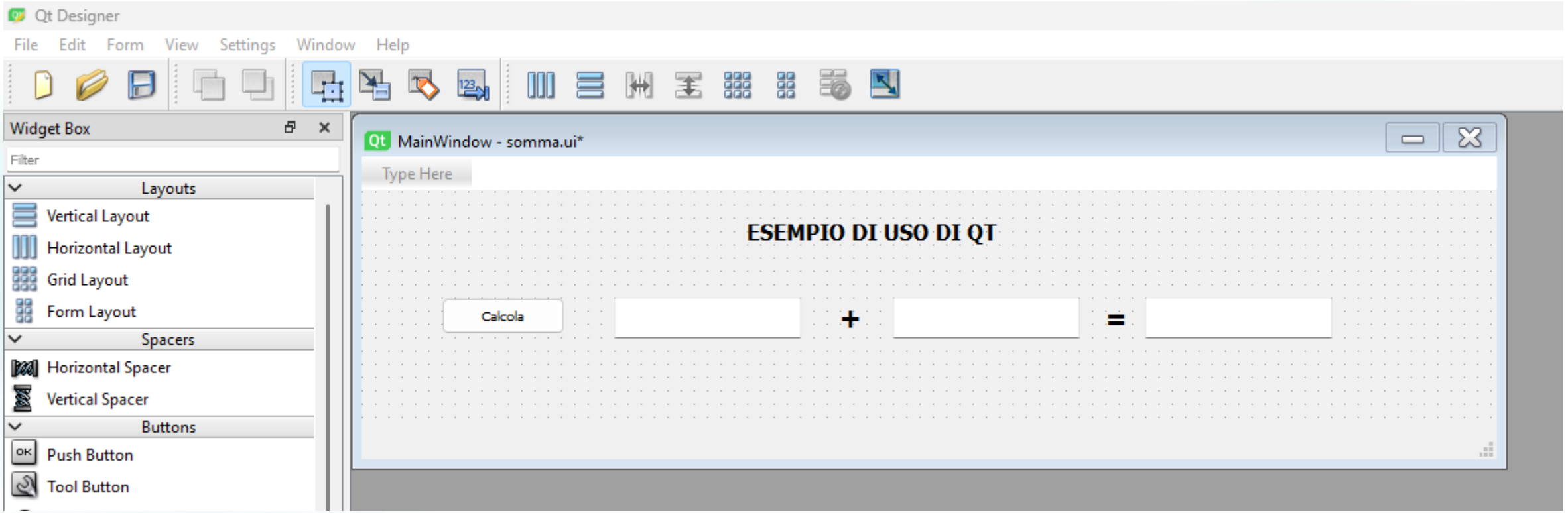
La classe «mainGUI.py» serve per creare un'**architettura indipendente** dalle modifiche della GUI («somma.ui») operate con QT Designer.

Progetto, utilizzando come IDE VS-Code:



Esempio: QT Designer

Progettare la GUI:



Esempio: trasformazione e logica-GUI

Esempio completo su GitHub:

https://github.com/penserini/ITS_AI_Specialist/tree/main/QT_Somma

somma_ui.py

```python
class Ui_MainWindow(object):
    def setupUi(self, MainWindow):
        if not MainWindow.objectName():
            MainWindow.setObjectName(u"MainWindow")
        MainWindow.resize(855, 228)
        self.centralwidget = QWidget(MainWindow)
        self.centralwidget.setObjectName(u"centralwidget")
        self.pushButton = QPushButton(self.centralwidget)
        self.pushButton.setObjectName(u"pushButton")
        self.pushButton.setGeometry(QRect(60, 80, 93, 28))
        self.num1 = QTextEdit(self.centralwidget)
        self.num1.setObjectName(u"num1")
        self.num1.setGeometry(QRect(190, 80, 141, 31))
        font = QFont()
        font.setPointSize(12)
        self.num1.setFont(font)
        self.label = QLabel(self.centralwidget)
        self.label.setObjectName(u"label")
        self.label.setGeometry(QRect(360, 80, 31, 31))
        font1 = QFont()
        font1.setPointSize(12)
        font1.setBold(True)
        self.label.setFont(font1)
        self.label.setTextFormat(Qt.RichText)
        self.num2 = QTextEdit(self.centralwidget)
        self.num2.setObjectName(u"num2")
        self.num2.setGeometry(QRect(400, 80, 141, 31))
        self.num2.setFont(font)
```

Esempio: logica indipendente

```python
if __name__ == "__main__":
    print("AVVIO GUI...")
    app = QApplication(sys.argv)
    window = MainGUI()
    window.show()
    sys.exit(app.exec())
```

mainGUI.py

```python
import sys
from PySide6.QtWidgets import QApplication, QMainWindow
from somma_ui import Ui_MainWindow

class MainGUI(QMainWindow):

    def __init__(self):
        super().__init__()
        self.ui = Ui_MainWindow()
        self.ui.setupUi(self)

        self.setWindowTitle("Semplice uso GUI con QT")

        #btn per fare la SOMMA
        self.ui.pushButton.clicked.connect(self.somma)

    # Funzione che svolge la somma al click del button
    def somma(self):
        print("AVVIO Somma...")
        self.num1testo = self.ui.num1.toPlainText().strip()
        self.num2testo = self.ui.num2.toPlainText().strip()
        somma = float(self.num1testo) + float(self.num2testo)
        self.ui.sum.setText(str(somma))
```

Overview

Parleremo di:

- Introduzione ai modelli LLM in pratica con TensorFlow
- Assistente AI con «Ollama»
- IoT con smart plugs: Shelly & Tapo
- Comandi vocali con «pyttsx3» e Ollama
- UI per Python con QT Designer



Creare un assistente AI con capacità di giocare ruoli diversi

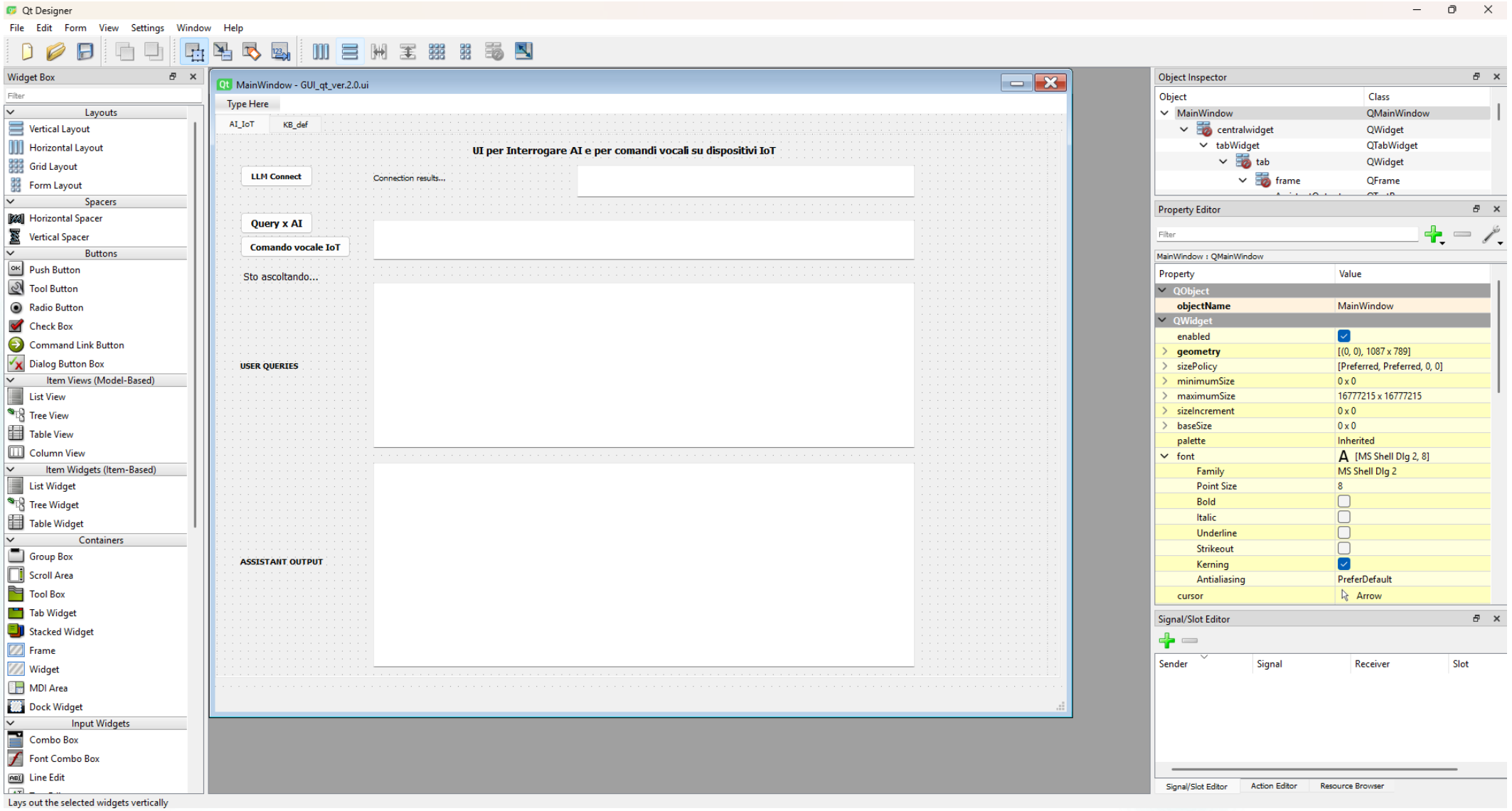
Project Work-5

Mettiamo assieme le competenze acquisite. Realizzare un Assistente AI con le seguenti caratteristiche:

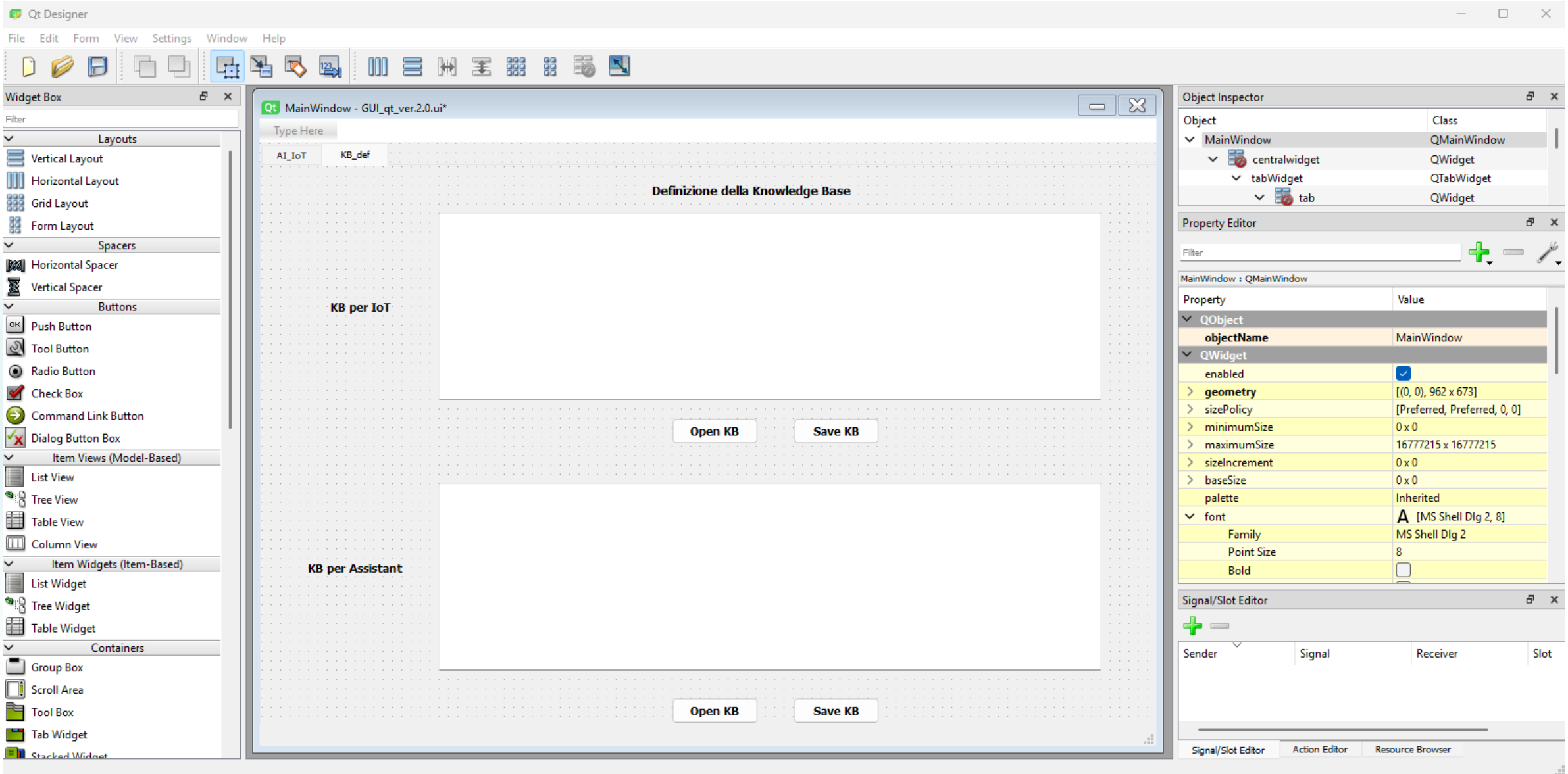
- Sia un esperto di Dispositivi IoT che accetta comandi vocali
- Sia anche in grado di rispondere a quesiti generici in base a come sia stata definita la knowledge base
- Utilizzi un modello LLM locale, es. Ollama
- Le diverse feature devono essere gestite attraverso una GUI in Python

Possibile implementzione: https://youtu.be/fpCjpc5aNLk

Soluz. PW-5: Progetto della GUI



Soluz. PW-5: Progetto della GUI



Soluz. PW-5

Esempio completo su GitHub e su Learn di UnivPM:

https://github.com/penserini/Lezioni_UnivPM/tree/main/Coding

https://learn.univpm.it

Bibliografia

[Raschka et al., book 2021] Sebastian Raschka, Vahid Mirjalili, «Machine Learning con Python – Nuova edizione – Costruire algoritmi per generare conoscenza», Apogeo. ISBN: 978-88-503-3524-4. 2021.

[Lambert, book 2024] Kenneth A. Lambert, «Programmazione in Python – Terza edizione», Apogeo education – Maggioli Editore. ISBN: 978-88-916-7143-1. 2024.

[Penserini, GitHub 2025] Loris Penserini, «Corso ITS Academy Turismo Marche: AI Specialist», https://github.com/penserini/ITS_AI_Specialist, 2025.